

East West University
Department of Computer Science and Engineering

CSE 488: LAB 03 (Handout)

Course Instructor: Dr. Mohammad Rezwanul Huq

Introduction to Apache Spark

Lab Objective

Introduce Apache Spark and its Python API (PySpark) as a high-performance, in-memory alternative to Hadoop MapReduce. Students will learn to create and manipulate Resilient Distributed Datasets (RDDs), apply a wide range of transformations and actions, and solve practical data processing problems using method chaining and lambda functions.

Lab Outcome

After completing this lab successfully, students will be able to:

1. Install and configure PySpark 4.0 on a Google Colab environment.
2. Explain the Apache Spark architecture and its key advantages over MapReduce.
3. Create RDDs using `parallelize()` and understand RDD immutability and lazy evaluation.
4. Apply narrow transformations: `map()`, `flatMap()`, and `filter()`.
5. Apply wide transformations: `reduceByKey()`, `groupByKey()`, `combineByKey()`, `join()`, and `cartesian()`.
6. Use action operations (`collect()`, `count()`, `distinct()`, `sortBy()`) to trigger execution.
7. Chain transformations to solve multi-step data processing problems in a single pipeline.

Psychomotor Learning Levels

This lab involves activities that encompass the following learning levels in the psychomotor domain.

Level	Category	Meaning	Keywords
P2	Manipulation	Reproduce activity from instruction or memory.	Copy, response, trace, Show, Start, Perform, Execute, Recreate.
P3	Precision	Execute skill reliably, independent of help.	Demonstrate, Complete, Show, Perfect, Calibrate, Control.
P4	Articulation	Adapt and integrate skills to satisfy a complex requirement.	Construct, Solve, Combine, Coordinate, Integrate, Adapt.

1. Apache Spark — Core Concepts and Architecture

1.1 What is Apache Spark?

Apache Spark is an open-source, distributed computing framework designed for fast, large-scale data processing. Originally developed at UC Berkeley's AMPLab in 2009, it overcomes the primary limitation of Hadoop MapReduce: the need to write intermediate results to disk after every MapReduce step. Spark keeps data in

memory across operations, making iterative algorithms (such as machine learning) and interactive analytics orders of magnitude faster.

Spark provides a unified platform for batch processing, streaming, SQL queries, machine learning, and graph computation — all using the same engine and the same high-level APIs.

Key Fact: Spark vs. MapReduce Speed

Spark can be up to 100x faster than Hadoop MapReduce for in-memory workloads and up to 10x faster for disk-based workloads. This gain comes from avoiding repeated disk I/O between pipeline stages and from its Directed Acyclic Graph (DAG) execution engine, which optimises the whole computation plan before running it.

1.2 Spark Architecture

A Spark application runs as a set of distributed processes coordinated by a central Driver Program. The architecture has three main components:

- **Driver Program:** The entry point of a Spark application. It runs the `main()` function, creates the `SparkContext/SparkSession`, converts user code into a DAG of tasks, and schedules those tasks on executors.
- **Cluster Manager:** Allocates resources across the cluster. Spark supports YARN (Hadoop), Mesos, Kubernetes, and its own Standalone manager. In this lab we use `local[*]` mode, which simulates a cluster on a single machine using all available CPU cores.
- **Executors:** Worker processes that run on cluster nodes. Each executor holds a portion of the data in memory (or disk) and executes the tasks assigned by the driver. Results are sent back to the driver after computation.

1.3 The SparkSession and SparkContext

In modern Spark (2.0+), `SparkSession` is the single unified entry point that replaces the older `SparkContext`, `SQLContext`, and `HiveContext`. It provides access to all Spark functionality.

The `SparkContext (sc)` is automatically available as `spark.sparkContext` and is required for creating low-level RDDs using `parallelize()`.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .master("local[*]") \
    .appName("Colab") \
    .getOrCreate()

sc = spark.sparkContext    # low-level RDD context
```

1.4 Resilient Distributed Datasets (RDDs)

An RDD is the fundamental data abstraction in Spark — an immutable, partitioned collection of elements that can be processed in parallel across a cluster. The name captures three key properties:

- **Resilient:** Fault-tolerant. If a partition is lost due to node failure, Spark can recompute it from the original data using the stored lineage (the sequence of transformations that created it).
- **Distributed:** Data is automatically split into partitions and distributed across executor memory. Each partition is processed by a separate task, enabling parallel computation.

- **Dataset:** A collection of records — numbers, strings, tuples, or any Python objects.

1.5 Lazy Evaluation and the DAG

Spark uses lazy evaluation: transformations (like `map`, `filter`) do not execute immediately. Instead, Spark builds a Directed Acyclic Graph (DAG) — a logical plan of all transformations — and executes them only when an action (like `collect()` or `count()`) is called.

This has two important benefits:

- Spark can optimise the full pipeline before execution (e.g., reorder or combine operations).
- No unnecessary work is done; intermediate results are not materialised unless required.

Transformation vs. Action

TRANSFORMATION: Returns a new RDD. Lazy — not executed until an action is called.

Examples: `map()`, `flatMap()`, `filter()`, `distinct()`, `union()`, `reduceByKey()`, `join()`

ACTION: Triggers execution of the DAG. Returns a result to the driver or writes to storage.

Examples: `collect()`, `count()`, `first()`, `take(n)`, `reduce()`, `saveAsTextFile()`

1.6 Narrow vs. Wide Transformations

Transformations are classified by whether they require data to be shuffled across the network:

- **Narrow Transformations:** Each partition of the parent RDD is used by at most one partition of the child RDD. No data shuffling required. Examples: `map()`, `flatMap()`, `filter()`. These are fast because data stays local.
- **Wide Transformations (Shuffle):** Multiple parent partitions may contribute to a child partition. Data must be redistributed across the network. Examples: `reduceByKey()`, `groupByKey()`, `join()`. These are slower but necessary for cross-partition aggregation.

2. Environment Setup (Re-run at Every Session)

Colab sessions reset between uses. Run the following setup cell at the beginning of every lab session before executing any other code.

```
# Install Java (required by Spark)
!apt-get update -qq
!apt-get install -y openjdk-17-jdk-headless

# Install PySpark 4.0
!pip install -q pyspark==4.0.0

# Set JAVA_HOME environment variable
import os
os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-17-openjdk-amd64"
os.environ["PATH"] = os.environ["JAVA_HOME"] + "/bin:" + os.environ["PATH"]

# Verify Java installation
!java -version
```

After setup, initialise the `SparkSession` and `SparkContext`:

```
from pyspark.sql import SparkSession
```

```

spark = SparkSession.builder.master("local[*]").appName("Colab").getOrCreate()
spark.sparkContext.setLogLevel("ERROR") # suppress verbose INFO logs

sc = spark.sparkContext # used to create RDDs

print(f"Spark Version : {spark.version}")
print(f"App Name      : {spark.sparkContext.appName}")
print(f"Master        : {spark.sparkContext.master}")

```

Note: If you see errors such as 'No module named pyspark' or a Java-related error, your Colab session has reset and you must re-run the setup cell.

3. RDD Transformation and Action Reference

3.1 Basic Transformations

Transformation	Example Input	Example Output	Description
map(f)	[1, 2, 3]	[1, 4, 9]	Apply function f to every element.
flatMap(f)	['a b', 'c']	['a', 'b', 'c']	Apply f and flatten one level.
filter(f)	[1, 2, 3, 4]	[2, 4]	Keep elements where f is True.
distinct()	[1, 2, 2, 3]	[1, 2, 3]	Remove duplicate elements.
union(rdd2)	[1, 2], [3, 4]	[1, 2, 3, 4]	Combine two RDDs.
intersection(r)	[1, 2, 3], [2, 3, 4]	[2, 3]	Elements common to both.
sortBy(f)	[3, 1, 2]	[1, 2, 3]	Sort by key function f.

3.2 Pair RDD (Key-Value) Operations

Pair RDDs are RDDs whose elements are (key, value) tuples. They support specialised operations for grouping, aggregating, and joining data by key.

Operation	Description
reduceByKey(f)	Merge values for each key using function f. Efficient — uses local combining before shuffle.
groupByKey()	Group all values for each key into a list. Less efficient than reduceByKey for aggregation.
mapValues(f)	Apply f only to the value of each (key, value) pair, leaving the key unchanged.
combineByKey(...)	Most general per-key aggregation: create combiner, merge value, merge combiners.
join(rdd2)	Inner join two key-value RDDs on their keys. Output: (key, (v1, v2)).

<code>cartesian(rdd2)</code>	Compute the Cartesian product of two RDDs. Output: all (a, b) pairs.
------------------------------	--

3.3 Actions

Action	Returns	Description
<code>collect()</code>	Python list	Retrieve all elements to the driver. Use only on small results.
<code>count()</code>	Integer	Return the number of elements in the RDD.
<code>first()</code>	Single element	Return the first element of the RDD.
<code>take(n)</code>	Python list	Return the first n elements.
<code>reduce(f)</code>	Single value	Aggregate all elements using function f.
<code>saveAsTextFile(path)</code>	None (side-effect)	Write RDD elements to a text file on disk.

4. Lab Activities

Lab Task 01: Word Count Using RDD Transformations

Implement the classic Word Count problem using PySpark RDDs. The pipeline demonstrates `parallelize()`, `flatMap()`, `map()`, and `reduceByKey()` — the Spark equivalent of a MapReduce job.

Data:

```
data = ["big data is powerful",
        "big data requires tools",
        "spark is fast"]
```

Algorithm:

- `parallelize()` — distribute the list across Spark partitions.
- `flatMap()` — split each sentence into individual words.
- `map()` — convert each word to a (word, 1) pair.
- `reduceByKey()` — sum up the counts for each word.
- `collect()` — bring results to the driver for display.

```
# Step 1: Create RDD from a Python list
data = ["big data is powerful",
        "big data requires tools",
        "spark is fast"]
rdd = sc.parallelize(data)

# Step 2: Split each line into words (flatMap flattens the nested lists)
rdd1 = rdd.flatMap(lambda line: line.split())
print(rdd1.collect())
# ['big', 'data', 'is', 'powerful', 'big', 'data', 'requires', 'tools', 'spark', 'is', 'fast']

# Step 3: Map each word to a (word, 1) key-value pair
rdd2 = rdd1.map(lambda word: (word, 1))
print(rdd2.collect())
```

```
# Step 4: Aggregate counts per key
rdd3 = rdd2.reduceByKey(lambda a, b: a + b)
print(rdd3.collect())
#
[('big', 2), ('data', 2), ('is', 2), ('powerful', 1), ('requires', 1), ('tools', 1), ('spark', 1), ('fast', 1)]
```

Note: `map()` vs. `flatMap()` — a critical distinction:

- **map(f):** Applies `f` to each element and returns one output per input. If `f` returns a list, you get a list of lists.
- **flatMap(f):** Applies `f` to each element and flattens the result by one level. Splitting a sentence into words with `map()` gives `[[word, word], [word]]` while `flatMap()` gives `[word, word, word]`. Always use `flatMap()` for tokenisation.

Filtering words by length — two equivalent approaches:

```
# Approach A: filter inside the flatMap lambda
rdd1 = rdd.flatMap(lambda line: [w for w in line.split() if len(w) >= 4])
print(rdd1.collect())
# ['data', 'powerful', 'data', 'requires', 'tools', 'spark', 'fast']

# Approach B: chain flatMap and filter (preferred — more readable)
rdd1 = rdd.flatMap(lambda line: line.split()).filter(lambda w: len(w) >= 4)
print(rdd1.collect())
```

Lab Task 02: `map()`, `filter()`, and `distinct()` — Numeric RDDs

Explore numeric RDD transformations to understand how `map()`, `filter()`, `distinct()`, and `sortBy()` compose into processing pipelines.

Part A — Cubes of Even Numbers:

```
rdd = sc.parallelize([1, 2, 3, 4, 5, 6])

# Chain: cube every number, then keep only even cubes
result = rdd.map(lambda x: x * x * x).filter(lambda a: a % 2 == 0)
print(result.collect()) # [8, 64, 216]
```

Part B — Circle Areas (map + filter):

Given a list of radii, compute each circle's area (πr^2) and keep only those exceeding 100.

```
import math
radius = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Step-by-step (explicit intermediate RDDs)
rdd = sc.parallelize(radius)
rdd1 = rdd.map(lambda r: math.pi * r * r)
rdd2 = rdd1.filter(lambda a: a > 100)
print(rdd2.collect())
# [113.097, 153.938, 201.062, 254.469, 314.159]

# Equivalent one-liner pipeline (method chaining)
result = sc.parallelize(radius) \
    .map(lambda r: math.pi * r * r) \
    .filter(lambda a: a > 100) \
    .collect()
print(result)
```

Part C — distinct() and sortBy():

```
rdd = sc.parallelize([1, 2, 2, 3, 3, 3])

# Remove duplicates then sort descending
result = rdd.distinct().sortBy(lambda x: x, ascending=False)
print(result.collect()) # [3, 2, 1]
print(result.count())   # 3
```

Lab Task 03: Set Operations — union(), intersection(), cartesian()

RDDs support set-like operations that combine data from two sources.

intersection() — Elements common to both RDDs:

```
rdd1 = sc.parallelize([1, 2, 3])
rdd2 = sc.parallelize([2, 3, 4])
print(rdd2.intersection(rdd1).collect()) # [2, 3]
```

union() + distinct() — Unique words from two paragraphs:

```
myrdd1 = sc.parallelize(['Hello World', 'Life is beautiful']) \
    .flatMap(lambda x: x.split())
myrdd2 = sc.parallelize(['Cruel World', 'Life is painful']) \
    .flatMap(lambda x: x.split())

# Union combines both; distinct removes duplicates
result = myrdd1.union(myrdd2).distinct()
print(result.collect())
# ['painful', 'Hello', 'World', 'is', 'Cruel', 'Life', 'beautiful']
```

cartesian() — All combinations of two RDDs:

```
rdd1 = sc.parallelize([1, 2])
rdd2 = sc.parallelize(["a", "b"])
print(rdd1.cartesian(rdd2).collect())
# [(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')]
```

Lab Task 04: Pair RDD Operations — join() for Common Word Frequencies

Find all words that appear in both of two paragraphs, and retrieve their occurrence count in each. This uses the join() operation on Pair RDDs — a distributed inner join by key.

Problem:

```
par1 = ['bangladesh is beautiful . bangladesh is not beautiful']
par2 = ['ewu is beautiful . ewu is painful']

# Expected output: words common to both paragraphs with their counts
# [('is', (2, 2)), ('beautiful', (2, 1))]
```

Pipeline — build a (word, count) Pair RDD for each paragraph, then join on the word key:

```
# Build word-count Pair RDD for paragraph 1
# filter(len > 1) removes punctuation characters like '.'
rdd1 = sc.parallelize(par1) \
    .flatMap(lambda x: x.split()) \
    .filter(lambda w: len(w) > 1) \
    .map(lambda x: (x, 1)) \
    .reduceByKey(lambda a, b: a + b)
```

```
# Build word-count Pair RDD for paragraph 2
rdd2 = sc.parallelize(par2) \
    .flatMap(lambda x: x.split()) \
    .filter(lambda w: len(w) > 1) \
    .map(lambda x: (x, 1)) \
    .reduceByKey(lambda a, b: a + b)

print(rdd1.collect())
# [('bangladesh',2), ('is',2), ('beautiful',2), ('not',1)]
print(rdd2.collect())
# [('ewu',2), ('painful',1), ('is',2), ('beautiful',1)]

# join() on the word key — only words in BOTH RDDs appear
rdd_common = rdd1.join(rdd2)
print(rdd_common.collect())
# [('is', (2,2)), ('beautiful', (2,1))]
```

Lab Task 05: groupByKey(), mapValues(), reduceByKey(), and combineByKey()

Compare different per-key aggregation strategies on a Pair RDD.

mapValues() — transform values without changing keys:

```
rdd = sc.parallelize([("a", 1), ("a", 2), ("b", 3), ("b", 4)])

# Attach a count of 1 to every value (sum+count pattern)
pair_rdd = rdd.mapValues(lambda v: (v, 1))
print(pair_rdd.collect())
# [('a', (1,1)), ('a', (2,1)), ('b', (3,1)), ('b', (4,1))]
```

groupByKey() — collect all values per key into a list:

```
rdd = sc.parallelize([("a", 1), ("a", 2), ("b", 3), ("b", 4)])

result = rdd.groupByKey().mapValues(list)
print(result.collect())
# [('b', [3,4]), ('a', [1,2])]

# Compute per-key average using groupByKey (less efficient)
import builtins
rdd = sc.parallelize([("a",1), ("a",2), ("b",3), ("b",2), ("a",12)])
result = rdd.groupByKey().mapValues(lambda x: builtins.sum(x) / len(x))
print(result.collect()) # [('b', 2.5), ('a', 5.0)]
```

reduceByKey() — efficient per-key aggregation:

```
rdd = sc.parallelize([("a",1), ("b",1), ("a",2), ("b",5), ("a",3)])

# Sum values per key (performs local combining before shuffle)
result = rdd.reduceByKey(lambda a, b: a + b)
print(result.collect()) # [('b', 6), ('a', 6)]
```

combineByKey() — most general per-key aggregation (average example):

`combineByKey()` is the most powerful aggregation operation. It takes three functions: `createCombiner` (initialises an accumulator when a key is seen for the first time), `mergeValue` (adds a value to an existing accumulator), and `mergeCombiners` (combines two accumulators from different partitions).

```
rdd = sc.parallelize([("a",1), ("a",2), ("b",3), ("b",2), ("a",12)])

combined = rdd.combineByKey(
```

```

    lambda value: (value, 1),                                # createCombiner
    lambda acc, value: (acc[0]+value, acc[1]+1),             # mergeValue
    lambda acc1, acc2: (acc1[0]+acc2[0], acc1[1]+acc2[1])     # mergeCombiners
)
print(combined.collect())
# [('b', (5,2)), ('a', (15,3))]  -- (sum, count) per key

# Compute final averages
averages = combined.mapValues(lambda x: x[0] / x[1])
print(averages.collect()) # [('b', 2.5), ('a', 5.0)]

```

reduceByKey vs. groupByKey

Prefer reduceByKey() over groupByKey() for aggregation tasks.

groupByKey() shuffles ALL values across the network before aggregating — very expensive on large data.

reduceByKey() performs local combination on each partition first, then shuffles only the partial results — much less network traffic, especially when many values share the same key.

Lab Task 06: Letter Frequency — End-to-End Pipeline

Count the frequency of each uppercase letter across a set of strings. This task demonstrates a complete multi-step pipeline using character-level processing.

```

# Step 1: Create RDD
rdd = sc.parallelize(["Big Data is Powerful", "Spark is FAST"])

# Step 2: Extract only alphabetic characters, convert to uppercase
letters = rdd.flatMap(lambda line: [ch.upper() for ch in line if ch.isalpha()])

# Step 3: Map each letter to (letter, 1)
letter_pairs = letters.map(lambda ch: (ch, 1))

# Step 4: Count occurrences per letter, sort by frequency descending
letter_counts = letter_pairs.reduceByKey(lambda a, b: a+b) \
    .sortBy(lambda x: x[1], ascending=False)

# Step 5: Collect and print
print(letter_counts.collect())
# [('A', 4), ('S', 4), ('I', 3), ('R', 2), ('F', 2), ('T', 2), ('P', 2), ...

```

East West University
Department of Computer Science and Engineering

CSE 488: LAB 03 (Exercise)

Course Instructor: Dr. Mohammad Rezwanul Huq

Complete all tasks in a single Colab notebook. Record all output using screenshots and answer all written questions in a separate .docx submission document.

Lab Task # 01 (RDD Creation and Basic Transformations):

1.(a). Create a SparkSession and SparkContext. Verify that Spark version 4.0 is running. Take a screenshot of the printed output showing Spark Version, App Name, and Master.

1.(b). Create an RDD from the following list and call collect() to display its contents. Then confirm its type using type().

```
data = ["hello world", "pyspark is great", "big data is everywhere"]
```

i. What Python type does collect() return?

ii. What is the Spark class name displayed by type()? What does this tell you about the RDD?

1.(c). Apply flatMap() to split the RDD into individual words, then apply filter() to keep only words with 4 or more characters. Print the result.

iii. Explain the difference between map() and flatMap() in your own words. Give one scenario where using map() instead of flatMap() would produce an incorrect or unexpected result.

Lab Task # 02 (Numeric Pipelines and Circle Areas):

2.(a). Run the cube-of-even-numbers pipeline on sc.parallelize([1,2,3,4,5,6]). Save the output and take a screenshot.

2.(b). Given radius = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], compute the circle area (πr^2) for each radius using map(), then keep only areas greater than 100 using filter(). Print the result.

iv. Write the same pipeline as a single chained expression (one line). Print the same result. Take a screenshot.

2.(c). Modify the pipeline to instead keep areas between 50 and 200 (inclusive). Name this pipeline filteredAreas and print its results.

v. How many elements remain? What is the minimum and maximum area in this range?

Lab Task # 03 (Set Operations):

3.(a). Using sc.parallelize(), create two RDDs:

```
rdd_a = sc.parallelize(["spark", "hadoop", "hive", "kafka"])
rdd_b = sc.parallelize(["spark", "kafka", "flink", "storm"])
```

Run intersection(), union().distinct(), and cartesian() on these RDDs. Print all three results.

vi. What does cartesian() return? How many elements does it produce for these two RDDs? Explain why.

3.(b). Explain what would happen if you called intersection() on two RDDs that share no elements. Write a brief code snippet to verify your answer and take a screenshot.

Lab Task # 04 (Pair RDD Operations and Join):

4.(a). Run the common-word-frequency pipeline on the two paragraphs below. Save the final joined output and take a screenshot.

```
par1 = ['data science is exciting . data engineering is critical']
par2 = ['machine learning is exciting . data is everywhere']
```

vii. What is the output of `rdd1.join(rdd2)`? Interpret each element of the result: what do the two numbers in each tuple represent?

viii. What SQL join type does RDD `join()` implement? What would `leftOuterJoin()` produce that is different?

4.(b). Modify the pipeline to also print the total count of common words found (use `count()` on the joined RDD).

Lab Task # 05 (groupByKey vs. reduceByKey vs. combineByKey):

5.(a). Given the RDD below, compute the per-key average using all three methods: `groupByKey()`, `reduceByKey()` with the `sum+count` pattern, and `combineByKey()`. Print results for all three.

```
rdd = sc.parallelize([("CSE", 75), ("CSE", 85), ("EEE", 90),
                     ("EEE", 80), ("CSE", 95), ("EEE", 70)])
```

ix. Do all three methods produce the same result? Why should `reduceByKey()` or `combineByKey()` be preferred over `groupByKey()` for large datasets?

5.(b). Add the following record to the RDD and re-run all three pipelines:

```
("ME", 88)
```

x. How many keys appear in the output now? Does each method handle the new key correctly?

Lab Task # 06 (End-to-End Pipeline — Letter Frequency):

6.(a). Run the letter-frequency pipeline on the strings below. Take a screenshot of the sorted output.

```
strings = ["Apache Spark is a Unified Analytics Engine",
           "It handles Batch Streaming SQL Machine Learning and Graph"]
```

xi. Which letter appears most frequently? How many unique letters appear in the output?

6.(b). Modify the pipeline to count lowercase letters separately (do not convert to uppercase). Print the top 5 most frequent lowercase letters using `take(5)` after sorting.

xii. Explain why `sortBy(lambda x: x[1], ascending=False)` is used instead of simply calling `sort()`. What does the lambda function extract, and why is `ascending=False` needed?

Submission

Submit your completed Colab notebook (.ipynb) and a document (.docx) containing screenshots for every task and answers to all written questions.

Name your files as per the following format:

- **Notebook:** 2022-1-60-001_CSE488_LAB03.ipynb
- **Document:** 2022-1-60-001_CSE488_LAB03.docx

Submit via the submission link provided in Google Classroom before the next lab session.